

AI Product *Kickoff Sprint*

Tìm vấn đề thật, thiết kế cho lúc AI sai, cắt một lát nhỏ đủ build, rồi dùng Day 06 để chứng minh bằng prototype và demo.

Day 5 khởi động dự án, Day 6 là mini-hackathon

Mini-hackathon bắt đầu từ 16:00 hôm nay

DAY 5 · FIND / DECIDE / PREP

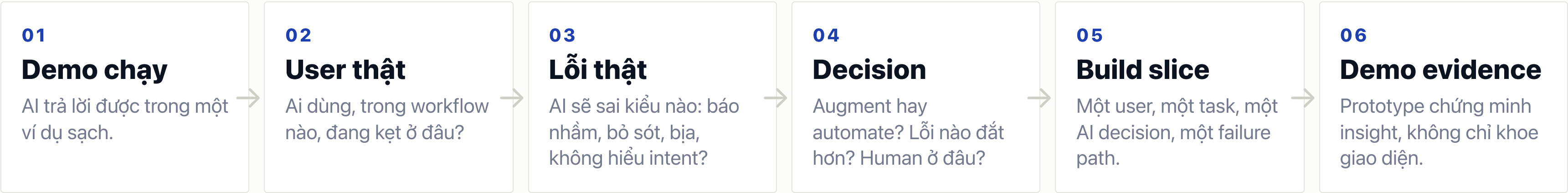
09:00	Vì sao AI product khác software thường: uncertainty, lỗi có hậu quả, UX phải có recovery.
10:00	Mổ app thật: MoMo Moni, Vietnam Airlines NEO, V-App/V-AI. Từ bug → decision.
13:00	Research và synthesis: evidence → insight → opportunity → build slice.
16:00	Bắt đầu mini-hackathon: chọn track/app, research nhanh, thin SPEC, owner plan.
23:59	Nộp evidence pack + thin SPEC draft.

DAY 6 · BUILD / TEST / SHOW

M1	Evidence gate: user thật, pain thật, build slice đủ nhỏ.
M2	Show một thứ chạy được: input → AI → output.
M3	Freeze scope + final SPEC draft.
M4	Dry run + backup demo.
M5/M6	Demo zone, peer feedback, top teams.
23:59	Nộp 1 repo cá nhân: group/ + individual/ .

Thông điệp vận hành: từ 16:00 Day 5, lớp không còn nghe lecture dài nữa. Mọi framework phải đổ về artifact: evidence, thin SPEC, prototype, demo.

Câu chuyện hôm nay: từ demo chạy được đến product đáng tin



Narrative spine: Day 5 không dạy product management chung chung. Day 5 dạy cách khởi động một dự án AI đủ thật để Day 6 build và demo.

HOOK

**Agent chạy được rồi.
Vậy tại sao có thể *không ai dùng?***

DEMO LÀ KHẢ NĂNG KỸ THUẬT · PRODUCT LÀ GIÁ TRỊ TRONG BỐI CẢNH THẬT

Khi AI đúng “gần đủ” nhưng product vẫn gãy

Failure không chỉ nằm ở model, mà ở trách nhiệm, UX và khả năng sửa output

GOOGLE BARD

Một câu sai có giá trị trường

Demo AI có thể tạo niềm tin quá nhanh. Một factual error trước công chúng trở thành rủi ro thương hiệu, cổ phiếu và niềm tin.

GAMMA / SLIDE AI

Tạo được nhưng sửa không được

0→80% rất nhanh. Nhưng nếu 20% cuối phải regenerate, sửa layout, sửa dữ liệu, user quay lại công cụ cũ.

AIR CANADA

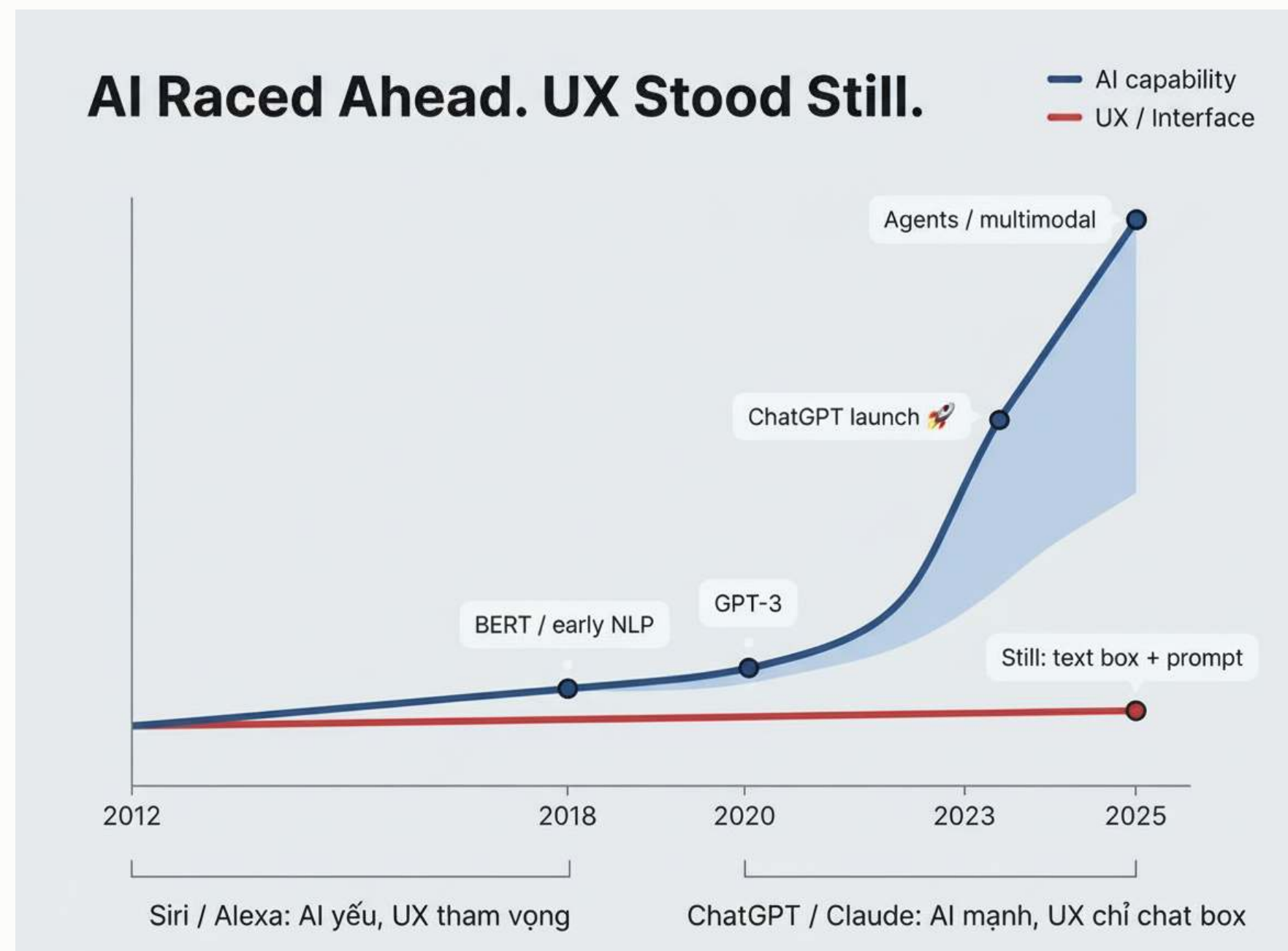
Bot sai, công ty chịu trách nhiệm

Chatbot bịa chính sách hoàn tiền. Bài học product: bot trên website là một phần của hệ thống chịu trách nhiệm.

Insight: cùng công nghệ AI, sản phẩm khác nhau ở nơi đặt ranh giới, cách sửa output và ai chịu trách nhiệm khi AI sai.

Vấn đề không phải AI yếu.

Vấn đề là ta đang đối xử AI như phần mềm thường.



Ghi nhớ Model ngày càng mạnh, nhưng product chỉ hữu ích khi interface, workflow và accountability giúp user khai thác đúng năng lực đó.

BLOCK 1

AI product bắt đầu từ *uncertainty*

INPUT MỜ HỒ · OUTPUT CÓ XÁC SUẤT · PROCESS KHÓ NHÌN THẤY

Phần mềm truyền thống sửa bug. AI product quản trị phân phối lỗi.

Software truyền thống	Input ổn định → logic xác định → output đúng/sai rõ. Khi bug được fix, lỗi đó kỳ vọng biến mất.
AI product	Input, output và process đều có phương sai. Cùng một yêu cầu có thể cho kết quả khác nhau theo model, context và dữ liệu.
Product implication	Không chỉ viết tính năng. Phải thiết kế ngưỡng chấp nhận, fallback, correction, logging và người chịu trách nhiệm.

Câu chốt: AI product không cố hứa “không bao giờ sai”; nó hứa “khi không chắc hoặc sai, hệ thống vẫn dẫn user đi đúng hướng”.

AI bất định ở ba lớp: input, output và process

INPUT UNCERTAINTY

User hỏi rất bần

- Thiếu context
- Dùng từ mơ hồ
- Đổi ý giữa chừng
- Cố tình prompt injection

OUTPUT UNCERTAINTY

Câu trả lời không cố định

- Cùng intent có nhiều cách trả lời
- Model update làm đổi style
- RAG/tool trả dữ liệu khác

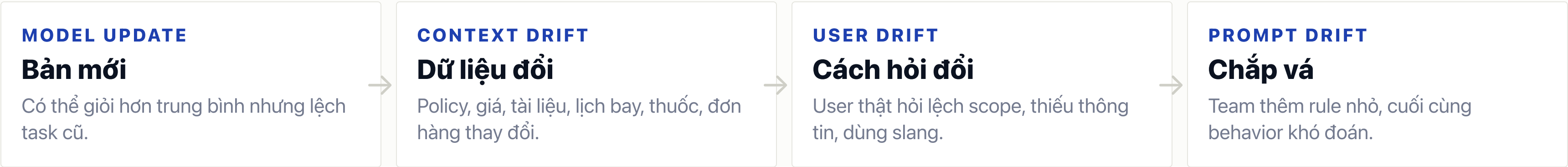
PROCESS UNCERTAINTY

Khó thấy vì sao

- Model tự suy luận
- Tool chain nhiều bước
- User khó biết nguồn đúng/sai

Thiết kế đúng: biến uncertainty thành quyết định product: hỏi lại lúc nào, hiện nguồn ở đâu, chuyển người khi nào, log correction ra sao.

Ngay cả khi không đổi code, product vẫn có thể đổi hành vi



Vì vậy AI product cần eval, version log và fallback, kể cả khi prototype Day 6 mới là bản mỏng.

Spam filter: cùng là sai, nhưng hậu quả khác nhau

FALSE POSITIVE

Email thật bị đưa vào spam

- User không thấy mail quan trọng
- Có thể mất cơ hội, lịch hẹn, hóa đơn
- Cần undo, whitelist, review folder

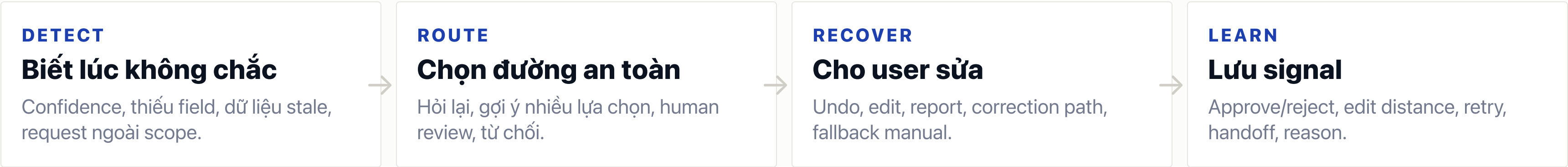
FALSE NEGATIVE

Spam lọt vào inbox

- User thấy và xóa được
- Gây phiền nhưng recover dễ hơn
- Rủi ro tăng nếu phishing/lừa đảo

Product decision: trước khi tối ưu model, phải trả lời sai kiểu nào tệ hơn với user, và hệ thống cho user recover thế nào.

AI product không xóa hết lỗi. Nó thiết kế đường đi cho lỗi.



Nếu prototype chỉ có happy path, đó chưa phải AI product. Ít nhất phải show một path khi AI không chắc hoặc sai.

BLOCK 2

Automation hay augmentation là quyết định *product*

QUYỀN HÀNH ĐỘNG · ACCOUNTABILITY · LEARNING SIGNAL

Automation và augmentation không phải hơn-kém

AUGMENTATION

AI tăng năng lực con người

- AI gợi ý, tóm tắt, draft, xếp hạng
- Người quyết định cuối
- Rủi ro thấp hơn, học từ approve/reject

AUTOMATION

AI tự hành động trong phạm vi đã định

- AI quyết định hoặc thực thi bước cuối
- Cần threshold, fallback, logging
- Sai khó undo thì rủi ro tăng mạnh

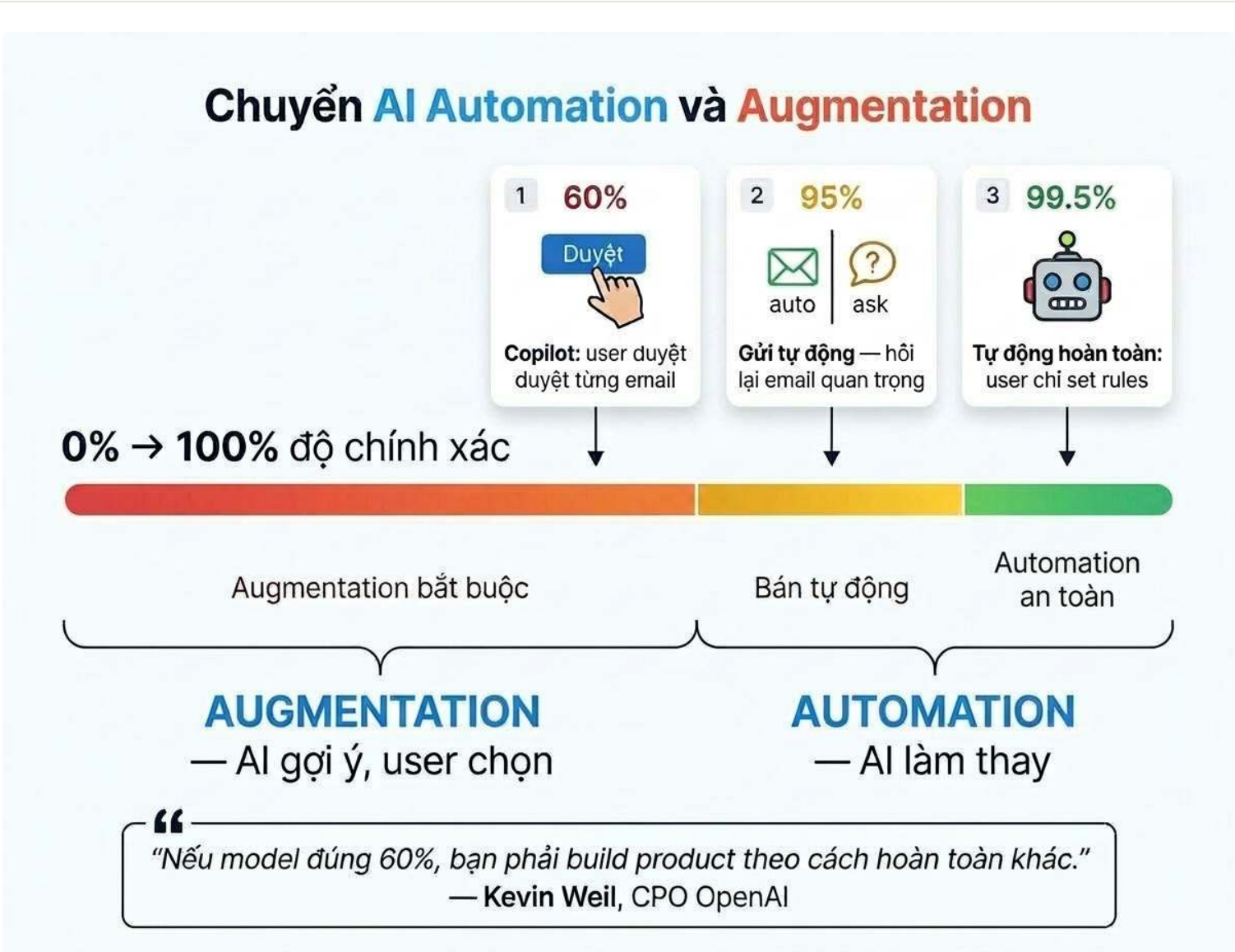
Insight: augmentation không phải bản kém của automation. Nó thường là bước đúng để giảm rủi ro, thu dữ liệu thật và học trước khi tăng tự động hóa.

Đừng hỏi “product này automate được không?” Hãy tách thành task.

FAQ deadline	Task hẹp, câu trả lời ổn định, nguồn rõ. Có thể conditional automation.
Debug project	Task mở, nhiều context, dễ dẫn nhóm đi sai. Nên augment coach bằng summary + suggested next step.
Chấm rubric	Rủi ro công bằng/accountability cao. AI hỗ trợ checklist, người quyết cuối.
Routing câu hỏi	AI phân loại và ưu tiên queue; case không chắc phải để người xem lại.

Cách dùng cho hackathon: chọn một workflow thật, tách 4-6 task nhỏ, rồi chỉ build một task AI có giá trị nhất.

Một task có thể đi qua nhiều mức tự động hóa



ĐIỂM NHÌN PRODUCT

Không có mốc accuracy chung cho mọi domain.

60% có thể vẫn hữu dụng nếu user chỉ cần duyệt; 99.5% vẫn nguy hiểm nếu AI tự động hành động sai.

CÁCH TĂNG AUTOMATION

Tăng quyền hành động sau khi có signal thật.

Approve/reject, correction log, case bị handoff và lỗi lặp lại là dữ liệu để nâng mức tự động hóa.

GATE CHO HACKATHON

Nếu sai khó undo, đừng full-auto.

Prototype nên bắt đầu bằng augmentation, confirmation hoặc human review.

Human-in-the-loop phải có vai trò rõ, không chỉ “cho người duyệt”

REVIEWER Kiểm tra output AI draft, người approve / edit / reject.	DECIDER Quyết định cuối AI đưa options, người chịu trách nhiệm chọn.	TRAINER Tạo learning signal Correction, label, rank, reason đi vào eval set.	RESCUER Can thiệp khi gãy Low-confidence, safety risk, escalation, handoff.
--	---	---	--

SPEC check: nếu viết “human review” mà không nói human làm vai trò nào và output review đi đâu, thì chưa đủ.

NHÓM · 7 PHÚT

Chọn một feature AI trong project hackathon.

Đặt feature lên *automation ladder*

AI LÀM GÌ · HUMAN LÀM GÌ · RỦI RO · KHI NÀO TĂNG AUTOMATION

BLOCK 3

Ba trụ thiết kế *AI product*

REQUIREMENT · UX · EVAL

AI sẽ sai, nên ba thứ phải thiết kế khác

1 · REQUIREMENT

Không chỉ feature

Trước: "Click X → Y".

Giờ: "Hỏi X → Y khoảng 85%; dưới 60% thì hỏi lại user".

Spec phải có ngưỡng, lúc không chắc và failure behavior.

→ Sai thế nào là chấp nhận được?

2 · UX

Không chỉ màn hình đẹp

Trước: thiết kế cho lúc đúng.

Giờ: thiết kế cho lúc **sai**: user thấy sai, sửa được, và tin lại được.

Graceful failure + trust recovery là phần lỗi của product.

→ Sai thì user làm gì?

3 · EVAL

Không chỉ pass/fail

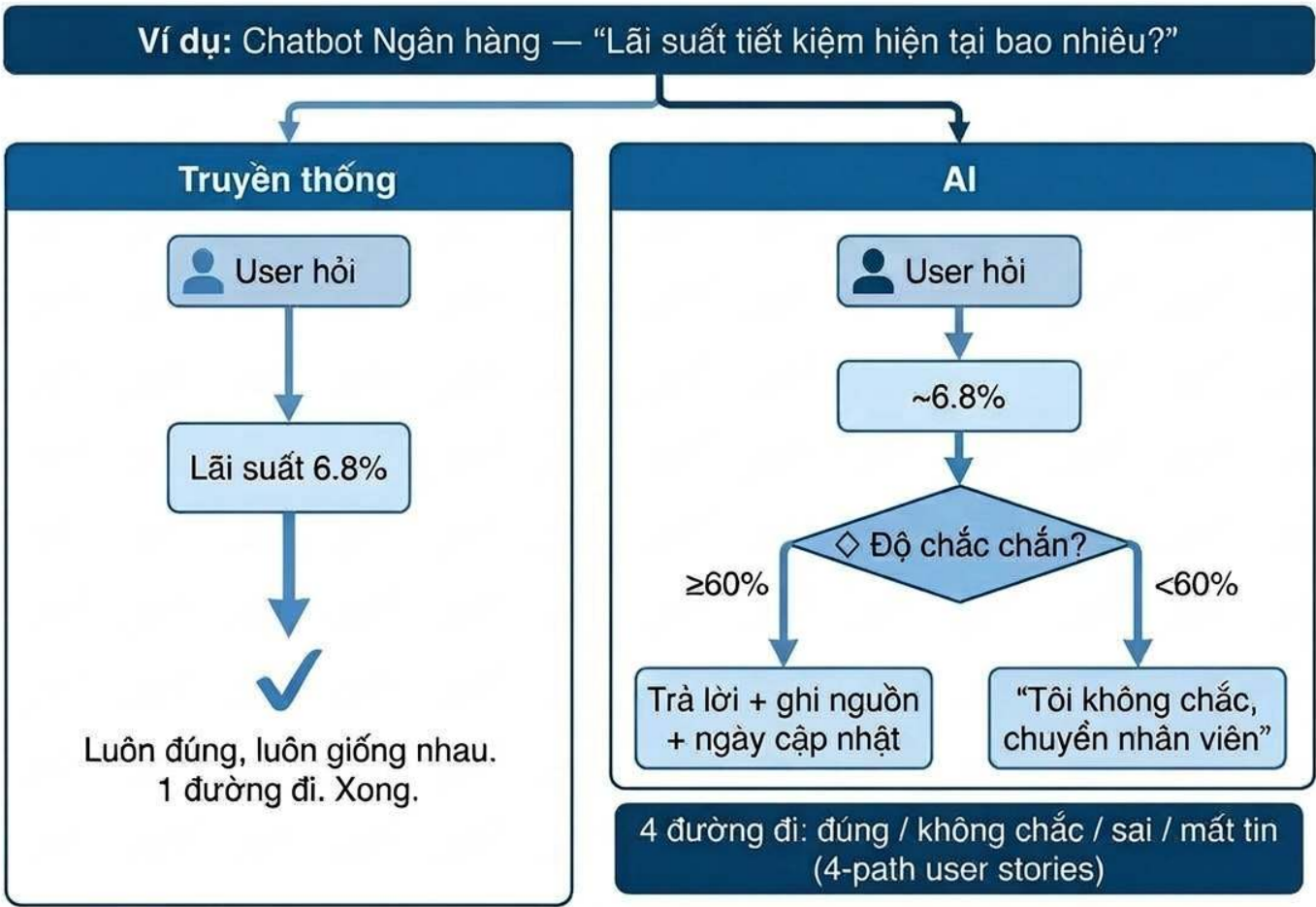
Trước: đạt / không đạt.

Giờ: chạy 100 lần → bao nhiêu % "đủ tốt", lỗi nào không được vượt ngưỡng?

PM quyết định quality distribution, không chỉ QA.

→ Bao nhiêu % sai là chấp nhận được?

AI requirement mô tả kết quả, ngưỡng và lúc sai thì sao



SPEC THƯỜNG

"User hỏi, chatbot trả lời" là chưa đủ để build.

Nó không nói nguồn, scope, ngưỡng chắc chắn, hay khi nào phải hỏi lại.

AI SPEC

Outcome + threshold + fallback.

Nếu đủ dữ liệu: trả lời có nguồn. Nếu thiếu hoặc dưới ngưỡng: hỏi lại/chuyển người.

CÂU HỎI THIẾT KẾ

Sai thế nào là chấp nhận được?

Requirement của AI luôn phải có lúc không chắc và lúc sai.

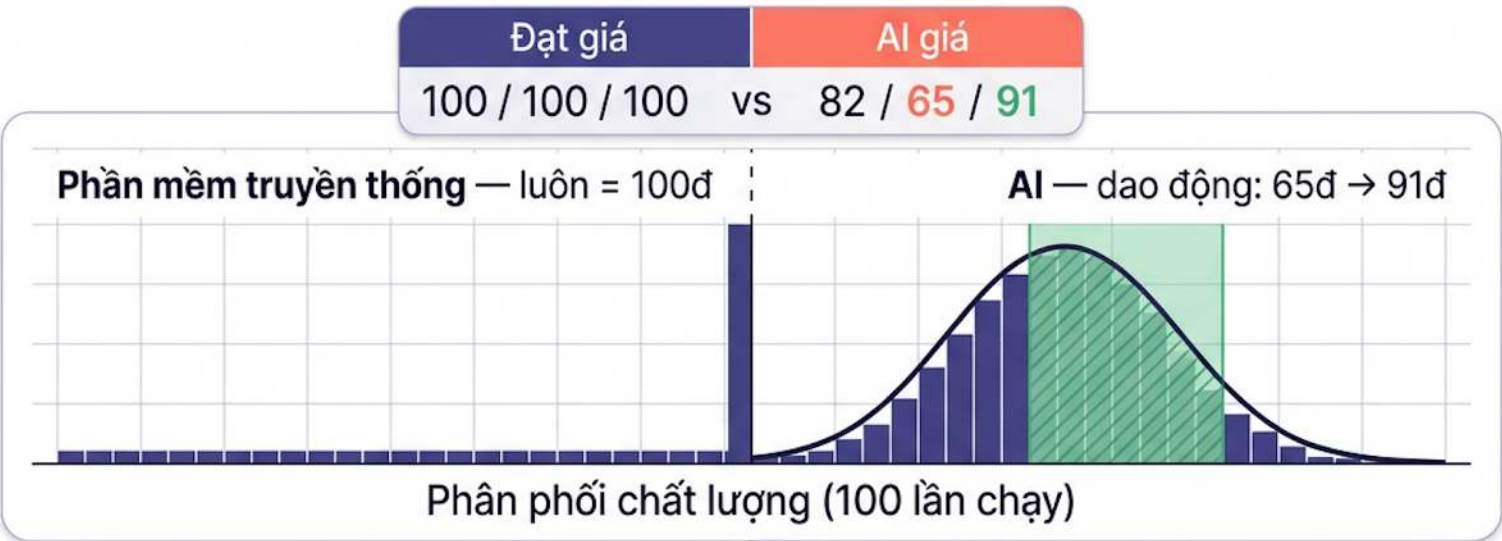
Trước khi viết feature, liệt kê cách product có thể sai

TRIGGER	HẬU QUẢ	MITIGATION
Hỏi ngoài phạm vi	Bot bịa chính sách hoặc điều khoản	Từ chối có kiểm soát, dẫn tới nguồn chính thức hoặc người thật
Input mơ hồ	Hiểu sai intent, trả lời đúng format nhưng sai nhu cầu	Hỏi lại, đưa lựa chọn, không tự đoán quá sâu
Dữ liệu cũ hoặc thiếu	Output tự tin nhưng sai thực tế	Hiện thời điểm cập nhật, confidence, fallback lookup
Prompt injection	User kéo bot ra khỏi vai trò hoặc policy	Instruction hierarchy, refusal, log red flag

Bài học Air Canada: vấn đề không chỉ là “bot cần thông minh hơn”, mà là product cần biết khi nào không được trả lời.

Eval không phải đạt/trượt. Eval là nhìn phân phối chất lượng.

Đánh giá (Eval): không phải đạt/trượt, mà sai bao nhiêu là chấp nhận được?



KHÔNG KẾT LUẬN TỪ MỘT DEMO ĐÚNG

AI tốt/xấu theo phân phối.

Chạy nhiều case: happy, mơ hồ, thiếu dữ liệu, prompt injection, edge case.

VIỆC CỦA PM/PRODUCT BUILDER

Phân loại lỗi trước khi sửa.

Lỗi nào do requirement, data/tool, UX, safety, hay eval case thiếu?

DAY05 RULE

Không drill công thức, nhưng phải biết lỗi nào đắt hơn.

Prototype ngày mai cần xử lý ít nhất một failure path thật.

Mỗi khi AI quyết định “có” hay “không”, nó có thể sai theo hai hướng

Hiểu rõ hai kiểu sai này trước, rồi mới nói đến cách đo

BÁO NHẦM · FALSE POSITIVE

AI nói “có” nhưng thật ra “không”

- Cảnh báo một thứ vốn bình thường
- Chặn nhầm nội dung hợp lệ, gây cản trở một giao dịch sạch

Thiệt hại: tạo việc thừa cho con người. Báo nhầm nhiều thì người dùng mất niềm tin và bắt đầu bỏ qua cảnh báo.

BỎ SÓT · FALSE NEGATIVE

AI nói “không” nhưng thật ra “có”

- Để lọt một thứ thật sự cần bắt
- Không phát hiện gian lận thật, bỏ qua nội dung độc hại

Thiệt hại: rủi ro bị để lọt. Người cần được bảo vệ thì không được, và lỗi biến mất khỏi tầm nhìn của người phụ trách.

Câu hỏi sản phẩm: cùng là “AI sai”, nhưng hai kiểu sai gây hai loại thiệt hại khác nhau. Với sản phẩm của mình, kiểu sai nào tốn kém hơn?

Hai thước đo, mỗi thước đo soi một kiểu sai

Precision soi báo nhầm; Recall soi bỏ sót

PRECISION · ĐỘ CHÍNH XÁC CỦA CẢNH BÁO

Khi AI báo "có", tin được bao nhiêu?

- Số báo đúng chia cho tổng số lần AI báo "có"
- Thấp nghĩa là báo nhầm nhiều

Precision thấp thì người dùng mất niềm tin vào cảnh báo và dần bỏ qua.

RECALL · ĐỘ BAO PHỦ

Trong các ca thật sự "có", bắt được bao nhiêu?

- Số bắt được chia cho tổng số ca thật sự cần bắt
- Thấp nghĩa là bỏ sót nhiều

Recall thấp thì những ca thật sự cần xử lý bị để lọt.

Ví dụ cho dễ hình dung: AI quét 1.000 giao dịch, báo "đáng ngờ" 40 lần và đúng 30 → $\text{precision} = 30 / 40 = 75\%$. Thực tế có 50 giao dịch xấu, AI bắt được 30 → $\text{recall} = 30 / 50 = 60\%$.

Lỗi nào đắt hơn sẽ quyết định prototype phải xử lý path nào

Báo nhầm đắt hơn

Prototype cần confirmation, source, confidence, human review hoặc undo rõ.

Bỏ sót đắt hơn

Prototype cần cảnh báo sớm, hỏi thêm, escalation hoặc checklist bắt buộc.

Cả hai đều đắt

Không automate vội. Dùng augmentation: AI đề xuất, người quyết, log correction.

Cả hai đều nhẹ

Có thể cho AI thử nhiều hơn, nhưng vẫn cần cách user sửa nhanh và report lỗi.

Mỗi lĩnh vực có cái giá của lỗi khác nhau

Đọc theo: báo nhầm là gì · bỏ sót là gì · lỗi nào đắt hơn → nên ưu tiên gì

LĨNH VỰC	BÁO NHẦM — FALSE POSITIVE	BỎ SÓT — FALSE NEGATIVE	NÊN ƯU TIÊN
Lọc nội dung cho trẻ em	Chặn nhầm nội dung lành mạnh — gây khó chịu nhưng sửa được	Để lọt nội dung độc hại — gây hại thật cho trẻ	Recall: đừng bỏ sót
Gợi ý viết code	Gợi ý sai — lập trình viên bỏ qua hoặc sửa, cái giá thấp	Bỏ lỡ một gợi ý hữu ích — chỉ là mất một cơ hội	Báo nhầm rẻ → recall cao chấp nhận được
Đọc phim X-quang	Báo nghi ngờ nhầm — bác sĩ xem lại, tốn công nhưng an toàn	Bỏ sót dấu hiệu bệnh — bác sĩ không bao giờ xem lại được	Recall: đừng bỏ sót
Duyệt khoản vay	Duyệt nhầm người rủi ro — mất tiền thật	Từ chối nhầm khách hàng tốt — mất doanh thu và công bằng	Tùy khẩu vị rủi ro, thường nghiêng precision

Không có đáp án chung: chọn ưu tiên precision hay recall là một quyết định sản phẩm, phụ thuộc lỗi nào gây hậu quả nặng hơn cho chính người dùng của mình.

Viết failure mode bằng câu đủ rõ để ngày mai build được

```
Nếu user [trigger],  
AI có thể [failure],  
hậu quả là [impact].
```

```
Prototype sẽ xử lý bằng [ask again / show source / human review / undo / fallback].  
Owner kiểm thử path này là [tên thành viên].
```

Gate: nếu chưa viết được câu này, nhóm chưa sẵn sàng build sâu.

4 câu mỗi AI product phải trả lời

1 · Khi đúng	User thấy gì? Copilot: gợi code màu xám, bấm Tab để chấp nhận.
2 · Khi không chắc	Hệ thống làm gì? Copilot: gợi ngắn hơn, ít tự tin hơn, user tự viết tiếp.
3 · Khi sai	User sửa thế nào? Copilot: tiếp tục gõ là gợi ý biến mất; cost gần 0.
4 · Khi mất tin	Gỡ thế nào? Copilot: tắt cho file/ngôn ngữ này, bật lại khi muốn.

Vì sao Copilot accuracy không cần hoàn hảo mà vẫn dùng được? Vì sai thì ít thiệt hại, sửa nhanh, và user giữ quyền quyết định. Microsoft Tay là phản ví dụ: không có recovery path khi bị user tấn công hành vi.

Khi AI sai, UX phải giảm thiệt hại và giữ niềm tin

GIẢM THIẾT HẠI

Xử lý lỗi nhẹ nhàng

- Đưa nhiều lựa chọn, không chỉ một đáp án tuyệt đối
- Cho user sửa output trực tiếp
- Fallback sang manual hoặc human review
- Ghi correction để biến lỗi thành signal

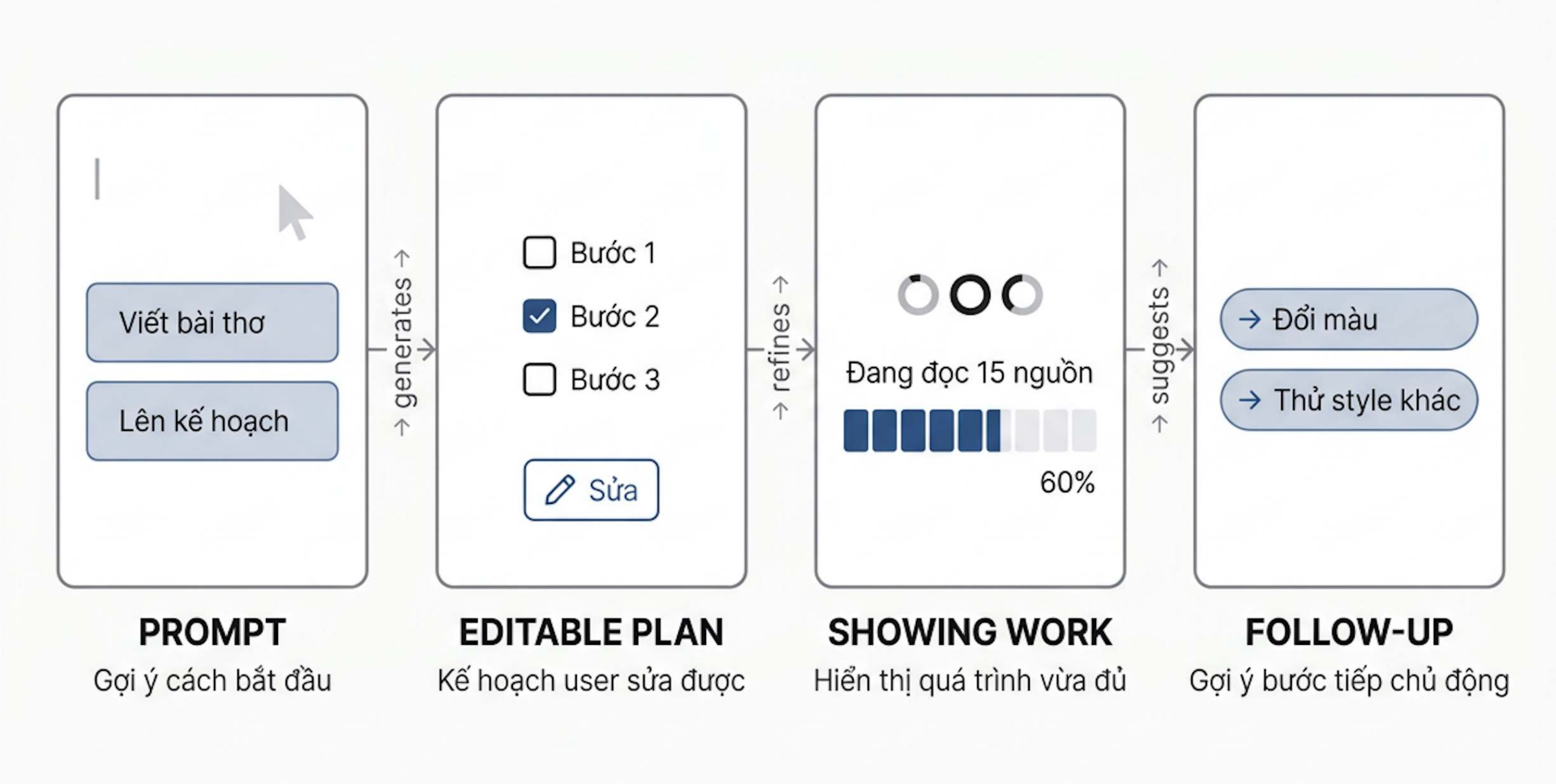
GIỮ NIỀM TIN

Giải thích và trao quyền

- **Grammarly:** giải thích lý do gợi ý
- **Kayak:** hiện confidence để user tự quyết
- Cho lựa chọn khác, undo, report hoặc tắt AI

Insight: graceful failure không phải câu “AI có thể sai”. Nó là cơ chế cụ thể để user thấy sai, sửa được, và quay lại tin sản phẩm.

Bốn thành phần giao diện mới cho AI



Aparna Chennapragada AI interface cần vừa đủ minh bạch: quá dài dòng thành cron job; quá ngắn thì user không biết AI đang đi đúng hướng không.

Mở sản phẩm AI thật: app → flow → path yếu → sửa

CHỌN MỘT APP ĐỂ DÙNG THỬ

MOMO

Moni

Trợ thủ tài chính,
phân tích chi tiêu,
chatbot trong app.

VIETNAM
AIRLINES

NEO

Chatbot hỗ trợ thông
tin chuyến bay, vé,
hành lý, khiếu nại.

V-APP

V-AI

Trợ lý ảo voice/text,
gợi ý theo context
trong app.

CÁCH LÀM TRONG 40 PHÚT

- 1 · DÙNG THỬ

Đọc promise, thử 2-3 query thật, chụp điểm gãy: kỳ vọng vs thực tế.
- 2 · VẼ FLOW

As-is: happy, low-confidence, failure, correction. Đánh dấu user kẹt ở đâu.
- 3 · SỬA MỘT PATH YẾU NHẤT

Đề xuất to-be: hỏi lại, source, button, undo, handoff, correction log.

Output: một sketch as-is/to-be + một câu product decision. Không kể bug rời rạc.

Bug chỉ là bằng chứng. Framework mới biến bug thành quyết định.

LAYER	CÂU HỎI	VÍ DỤ MONI / NEO
Promise	User kỳ vọng gì?	NEO trông như bot tra chuyển bay, nhưng cần mã chuyển bay.
Intent	AI hiểu đúng ý định không?	“Linh tinh” bị hiểu như keyword thay vì khái niệm mơ hồ.
Data / Tool	Có nguồn và tool đúng không?	Trả link chung, không tra được dữ liệu cần thiết.
Safety / Behavior	AI có hành vi rủi ro không?	Prompt injection, đồng ý với user dù dữ liệu mâu thuẫn.
UX Recovery	User recover thế nào?	Không typing indicator, duplicate response, không correction loop.

Mỗi finding phải được viết lại thành một product decision

```
Khi user [trigger],  
AI/product [failure],  
hậu quả là [impact],  
lỗi thuộc layer [taxonomy],  
nên sửa bằng [requirement / eval / UX / data / automation],  
đo bằng [metric hoặc signal].
```

Ví dụ: “linh tinh” là Intent + UX Recovery. Sửa bằng hỏi lại tiêu chí, đề xuất nhóm giao dịch nhỏ/chưa phân loại, đo bằng user correction rate.

Nếu không đi vào SPEC, bug sẽ chỉ là chuyện kể trên lớp

FINDING THẤY ĐƯỢC	NGHĨA PRODUCT THẬT SỰ	GHI VÀO SPEC NHƯ THẾ NÀO
Prompt injection	User có thể kéo AI ra khỏi vai trò. Đây không phải "bug vui", mà là lỗi boundary + safety.	Top failure mode: khi user yêu cầu bỏ qua policy, AI phải từ chối, giải thích phạm vi và handoff/log red flag.
Keyword "linh tinh"	AI hiểu chữ, nhưng không hiểu intent mơ hồ của user. Vấn đề nằm ở low-confidence path.	4 paths: với query mơ hồ, AI hỏi lại tiêu chí hoặc đưa 2-3 lựa chọn; thêm eval case cho input mơ hồ.
Không có trạng thái / typing indicator	User không biết AI đang xử lý, bị treo hay đã hỏng. Đây là lỗi trust + latency signal.	UX recovery: hiện trạng thái, bước đang làm, thời gian chờ, nút thử lại/hủy/chuyển người.
Kỳ vọng lệch / trả trùng	Promise của product không khớp khả năng thật; lỗi lặp lại chứng tỏ thiếu regression test.	Trust + Eval: viết rõ boundary ngay onboarding; thêm test case "không lặp câu trả lời" và metric report/correction.

Công thức debrief: đừng ghi "bot lỗi". Hãy ghi: **finding → layer → product decision → SPEC field → test/failure path.**

BLOCK 4

FIND → SYNTHESIZE → DECIDE

EVIDENCE · INSIGHT · OPPORTUNITY · BUILD SLICE

Ba cách tìm đúng user trước khi build

1 · TỰ TRẢI NGHIỆM

Self-use

Mở app, làm task thật, ghi lại moment mình kẹt. Nhanh nhất nhưng chỉ là một góc nhìn.

2 · TÌM USER THẬT

Social / phỏng vấn

Tìm nơi user than phiền: group, comment, diễn đàn, bạn bè. Hỏi “lần gần nhất bạn bị kẹt là khi nào?”.

3 · CÀO REVIEW

Review mining

Lấy 30-50 review App Store/Play, dùng AI gom nhóm than phiền, chọn top failure mode có bằng chứng.

Câu neo: bạn không phải lúc nào cũng là user thật. App ngân hàng cho người lớn tuổi, app y tế cho bệnh nhân mãn tính, app travel cho gia đình đi nghỉ đều có context khác sinh viên.

Research không dừng ở quote. Nó phải đổi quyết định build.



Không nhận: “AI assistant cho healthcare”. **Nhận:** “người mới khám không biết chọn chuyên khoa, AI hỏi 3 câu và gợi 2 chuyên khoa, red flag chuyển người”.

Mở rộng ý tưởng rồi đóng lại bằng evidence và scope

Diverge

Tạo nhiều hướng, không chốt ý đầu tiên, giữ cả ý lạ để học.

Cluster

Gom theo user pain hoặc workflow step, không gom theo tên feature.

Score

Evidence strength, user value, AI fit, build feasibility, failure risk.

Commit

Chọn một build slice; các ý còn lại đưa vào backlog, không build hôm nay.

TIÊU CHÍ	CÂU HỎI
Evidence	Có nguồn ngoài nhóm không?
AI fit	AI có tạo lợi thế so với rule/manual không?
Buildable	Trong một ngày có show được input → AI → output không?

Day05 chỉ cần SPEC đủ để build, không cần PRD đầy đủ

Evidence	Nhóm thấy pain thật ở đâu, nguồn nào, quote/screenshot nào?
Build slice	Một user, một task, một AI decision, một output.
Auto/Aug	AI gợi ý, chuẩn bị, hay tự hành động? Human làm gì?
4 paths	Happy, low-confidence, failure, correction.
Owner plan	Ai làm prompt/test, UI, repo, demo script, evidence?

AI Product Canvas là một trang giữ product không trôi về demo

VALUE

Cho ai, đau ở đâu?

User cụ thể, pain cụ thể, AI giải gì mà cách hiện tại chưa giải tốt.

TRUST

Khi AI sai thì sao?

User biết, sửa, undo, handoff và regain trust bằng cách nào.

FEASIBILITY

Có đáng build không?

Cost/request, latency, data, risk chính và ngưỡng kill.

Learning signal: user correction đi vào đâu? Product tốt lên bằng signal nào: approve, edit, retry, handoff, report sai?

User Stories trong Day05 là 4 paths, không phải một dòng happy path

PATH	CÂU HỎI CẦN TRẢ LỜI	VÍ DỤ QUYẾT ĐỊNH UX
Happy	AI đúng và tự tin, user thấy gì?	Gợi ý hiện rõ, accept một thao tác
Low-confidence	AI không chắc, có hỏi lại không?	Hiện 2-3 lựa chọn hoặc yêu cầu thêm thông tin
Failure	AI sai, user recover thế nào?	Undo, sửa trực tiếp, chuyển người thật
Correction	User sửa, data đi vào đâu?	Correction log, cập nhật rule/test set

Nếu không thu signal, product AI không tốt lên

1 · User correction đi vào đâu?

Sửa giao dịch, đổi nhãn email, reject gợi ý code, report câu sai: lưu thành dữ liệu học được.

2 · Signal nào cho biết tốt lên hay tệ đi?

Approve rate, edit distance, retry, handoff, time-to-resolution, report sai.

3 · Dữ liệu có marginal value không?

Model đã biết kiến thức chung. Product thắng bằng dữ liệu domain, user-specific và human judgment.

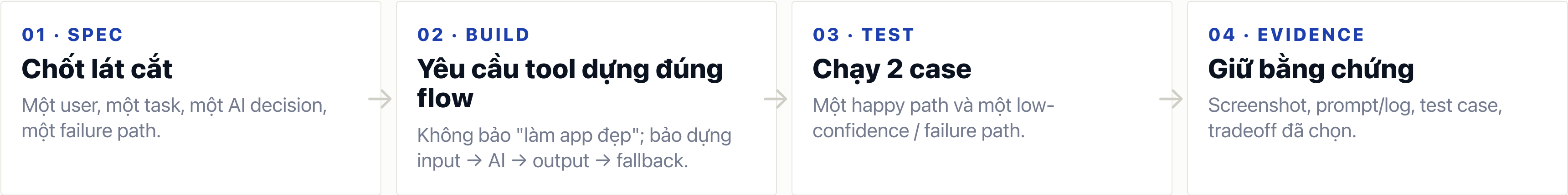
BLOCK 5

BUILD PREP

vibe code có kiểm soát

THIN SPEC · PROTOTYPE FLOW · FAILURE TEST · DEMO EVIDENCE

Vibe coding = biến SPEC thành prototype có bằng chứng



Ý đồ: dùng AI builder để hiện hình quyết định product, không dùng AI builder để nghĩ hộ product. Nếu prototype không chứng minh được failure path, demo đẹp cũng chưa đủ.

Cuối Day05, nhóm cần chuẩn bị đủ để sáng mai build ngay



Không cần tối nay build xong. Nhưng nếu thiếu 5 thứ này, sáng Day06 nhóm sẽ mất nửa ngày để chọn lại đề.

HACKATHON BRIEFING

Từ evidence đến *thin SPEC + prototype*

DAY05 + DAY06 CHẤM CHUNG 100 ĐIỂM

Hackathon này là gì, nhóm làm gì, nộp gì?

TRACK LÀ GÌ?

Một miền app thật dễ soi và cải tiến

Learning OS, Travel, Food Delivery, Finance, Healthcare. Track không phải scope; mỗi nhóm phải cắt thành một flow nhỏ.

NHÓM LÀM GÌ?

Research → SPEC → prototype

Tìm evidence thật, chọn một pain, quyết định AI augment/automate, thiết kế 4 paths, build demo đủ chứng minh.

TỐI DAY05

Evidence + thin SPEC

User, pain, build slice, failure path, owner plan.

DAY06

Build + test + dry run

Prototype có flow nhìn được, test happy + failure/low-confidence, chuẩn bị demo narrative.

CUỐI DAY06

Demo + repo cá nhân

Pitch vấn đề, evidence, prototype, failure handling, reflection; nộp repo có group/ và individual/.

Chọn domain trước, rồi cắt thành một build slice đủ nhỏ

TRACK	PRODUCT / APP	IDEA GỢI Ý
A • Learning OS (Vin AI Thực Chiến)	LMS hiện tại, Discord lớp	Cá nhân hóa lộ trình học, gia sư LMS tương tác, trợ lý lớp live, lab copilot.
B • Travel & Hospitality	Vinpearl, Sun World / SunGroup	AI concierge, gợi ý lịch trình, hỗ trợ đặt phòng/đổi hủy, hỏi đáp tiện ích.
C • Food & Local Delivery	ShopeeFood, GrabFood, BeFood, Xanh SM Ngon	Gợi ý món theo nhu cầu, xử lý đơn hàng lỗi, hỗ trợ hoàn tiền, tìm quán theo ngữ cảnh, tối ưu giao nhận.
D • Personal Finance	MoMo, ZaloPay, app ngân hàng	Phân loại chi tiêu, phát hiện giao dịch lạ, giải thích phí/giao dịch, nhắc thanh toán.
E • Healthcare	Vinmec, Long Châu, Pharmacity	Gợi ý chuyên khoa, giải thích đơn thuốc, nhắc tái khám/nhắc thuốc, hỗ trợ đặt lịch, tracking chỉ số sức khỏe.

Day05 + Day06 = 100 điểm

HẠNG MỤC	ĐIỂM	GHI CHÚ
SPEC milestone	25	Nhóm 20 + cá nhân 5
Prototype milestone	15	Nhóm 10 + cá nhân 5
Demo Day	25	Peer feedback + written comments + bonus top teams
UX Exercise	10	Cá nhân 8 + bonus vote 2
Individual form	25	Role 10 + reflection 15

Prototype có ba mức, nhưng phải chứng minh được AI behavior

LEVEL 1 Sketch • Flow rõ tất cả bước • Có prompt/test minh họa AI behavior • Tất cả thành viên giải thích được	LEVEL 2 Mock prototype • HTML, app hoặc Figma • Thể hiện output AI và fallback • Flow rõ + có edge case	LEVEL 3 Working prototype • Input → AI → output • Demo live được • Base 10 nếu chạy ổn và narrative rõ
---	--	---

Điểm quan trọng: prototype không cần full app, nhưng phải chứng minh được một AI decision và một path khi AI không chắc hoặc sai.

Mỗi học viên nộp một repo cá nhân có folder nhóm và cá nhân

```
Day06-MãHọcViên-HọVàTên-MãNhóm/  
├── group/  
│   ├── spec-final.md  
│   ├── prototype-readme.md  
│   ├── demo-slides.pdf  
│   └── prompt-tests-or-failure-log.md  
└── individual/  
    └── reflection.md
```

Submission gate: cuối Day 6 nộp 1 repo cá nhân public. Folder **group/** chứa artifact nhóm; folder **individual/** chứng minh đóng góp cá nhân.

Năm nguyên tắc sau Day05

